

Informe: Trabajo Práctico #4 - Módulos de kernel y llamadas a sistema

Asignatura: Sistemas de Computación

Institución: Facultad de Ciencias Exactas, Físicas y Naturales (FCEyN) – UNC

Docente: Javier Alejandro Jorge

Datos del Grupo y Repositorio

- **Integrantes:**
 - Macarena Vanina González
 - Marcos Nieto
 - Mario Pampiglione
- **Repositorio:** https://github.com/Maca040/SistComp_TeamMergeNoConflict.git

Modulos de Kernel

Un Módulo de Kernel Linux se define como un segmento de código capaz de cargarse y descargarse dinámicamente dentro del kernel según sea necesario. Estos módulos mejoran las capacidades del núcleo sin necesidad de reiniciar el sistema. Un ejemplo notable es el módulo de controlador de dispositivo, que facilita la interacción del núcleo con los componentes de Hardware vinculados al sistema.

En ausencia de módulos, el enfoque predominante se inclina hacia los núcleos monolíticos, que requieren la integración directa de nuevas funcionalidades en la imagen del núcleo.

Este enfoque da lugar a núcleos más grandes y requiere la reconstrucción del núcleo y el subsiguiente reinicio del sistema cuando se desean nuevas funcionalidades.

Para comenzar vamos a necesitar instalar algunas dependencias:

```
sudo apt-get install build-essential checkinstall kernel-package linux-source
```

DESAFIO #1

1)

a) ¿Qué es checkinstall y para qué sirve? Checkinstall es un programa de código abierto para sistemas operativos Unix-like que facilita la instalación y desinstalación de software compilado desde el código fuente, permitiendo que sea administrado por el sistema de gestión de paquetes nativo del sistema (APT/Dpkg en Debian/Ubuntu, RPM en Fedora/RHEL, o pkgtool en Slackware). A diferencia de `make install`, donde al ser ejecutado los archivos se dispersan por el sistema sin dejar registro de qué se instaló ni dónde, CheckInstall realiza un monitoreo de la instalación, es decir, intercepta el comando de instalación (como `make install` por ejemplo) y registra todos los archivos creados o modificados generando un paquete nativo (.deb, .rpm o

Slackware) con los archivos instalados. Esto permite desinstalar el software posteriormente usando el gestor de paquetes del sistema, sin tener que buscar archivos manualmente.

b) Crear un Hello_world y empaquetarlo

```

EXPLORER
TP4
  Hola_empaquetado
    hola
    hola.c
    makefile
  kernel-modules
  part1
  module
    .mimodulo.ko.cmd
    .mimodulo.mod.cmd
    .mimodulo.mod.o.cmd
    .mimodulo.o.cmd
    .Module.symvers.cmd
    .modules.order.cmd
    Makefile
    mimodulo.c
    mimodulo.ko
    mimodulo.mod
    mimodulo.mod.c
    mimodulo.mod.o
    mimodulo.o
    Module.symvers
    modules.order
    MOK.der
    MOK.priv
  syscalls
    copiar_archivo.c
    ejemplo_printf.c

C hola.c
M makefile

Hola_empaquetado > C hola.c > main()
1 #include <stdio.h>
2
3 int main() {
4     printf("TEAM_MERGE_NO_CONFLICT\n");
5     return 0;
6 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
mario@mario-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/Hola_empaquetado$ make
gcc -Wall -Wextra -O2 -o hola hola.c
mario@mario-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/Hola_empaquetado$ sudo checkinstall
[sudo] contraseña para mario:

checkinstall 1.6.3, Copyright 2010 Felipe Eduardo Sanchez Diaz Duran
Este software es distribuido de acuerdo a la GNU GPL

The package documentation directory ./doc-pak does not exist.
Should I create a default set of package docs? [y]:

```

```

mario@mario-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/Hola_empaquetado$ sudo checkinstall

*****

Done. The new package has been installed and saved to

/home/mario/Documentos/Ejercicios_Sist_Comp/TP4/Hola_empaquetado/hola-empaquetado_20260520-1_amd64.deb

You can remove it from your system anytime using:

dpkg -r hola-empaquetado

*****

mario@mario-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/Hola_empaquetado$ ls
description-pak hola hola.c hola-empaquetado_20260520-1_amd64.deb makefile
mario@mario-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/Hola_empaquetado$ cd /home
mario@mario-IdeaPad-Gaming-3-15IAH7:/home$ hola
TEAM MERGE NO CONFLICT
mario@mario-IdeaPad-Gaming-3-15IAH7:/home$ ls /usr/local/bin/
apt gnome-help highlight-mint search yelp
mario@mario-IdeaPad-Gaming-3-15IAH7:/home$ sudo dpkg -r hola-empaquetado
(Leyendo la base de datos ... 972737 ficheros o directorios instalados actualmente.)
Desinstalando hola-empaquetado (20260520-1) ...
mario@mario-IdeaPad-Gaming-3-15IAH7:/home$ ls /usr/local/bin/
apt gnome-help highlight-mint search yelp
mario@mario-IdeaPad-Gaming-3-15IAH7:/home$ hola
Orden «hola» no encontrada. Quizá quiso decir:
  la orden «lola» del paquete deb «lola (1.6-1)»
  la orden «cola» del paquete deb «git-cola (4.3.2-1)»
Pruebe con: sudo apt install <nombre del paquete deb>
mario@mario-IdeaPad-Gaming-3-15IAH7:/home$

```

Como se puede apreciar en las dos imágenes se arma el .o con make, luego se hace uso de checkinstall que en este ejemplo se encarga empaquetar en un .deb e instalar en /usr/local/bin/ según se especificó en el makefile. Al ejecutar hola posicionado en algún directorio (home en este caso) debería ser visible en consola la respuesta ("TEAM_MERGE_NO_CONFLICT") de nuestro programa instalado. "ls /usr/local/bin/" en este caso puntual es usado solo para mostrar el hecho de que existe "hola" una vez completada la instalación y desaparece tras la inmediata desinstalación efectuada por dpkg ("sudo dpkg -r hola-empaquetado"). Por último se intenta llamar nuevamente al programa para dejar claro que efectivamente este ya no está

disponible (es un agregado un poco más visual al hecho de haberse cerciorado de que ya no estaba en /usr/local/bin/).

c) Seguridad del Kernel

Rootkits: es un software malicioso que da acceso privilegiado al sistema mientras se oculta, modificando el kernel u otros componentes.

Dado que la seguridad del kernel de linux es importante para proteger sistemas contra amenazas como los módulos no firmados y rootkits, nombramos a continuación algunas de las medidas que podemos implementar para mejorar la seguridad del Kernel.

- **Firmar módulos del Kernel:** El kernel de linux permite extender sus funcionalidades mediante módulos, que son piezas de código cargadas dinámicamente. El hecho de tener módulos no firmados pueden representar riesgos ya que podrían contener algún tipo de malware.

En sistemas con **Secure Boot**, los módulos deben estar firmados con una clave privada y verificados con la clave pública

- **Habilitar Secure Boot:** Esto se encuentra disponible en firmware UEFI y requiere que todos los cargadores de arranque y módulos de kernel estén firmados con claves de confianza
- **Configurar module.sig_enforce:** esto se utiliza para rechazar los módulos sin firma.+

DESAFIO #2

2)

a) ¿Qué funciones tiene disponible un programa y un módulo? Un programa en espacio de usuario puede usar (y en general hace uso) de la biblioteca estándar de C, además de otras bibliotecas que el usuario pueda querer agregar y todas las definiciones de las funciones que se encuentren en el programa y hagan uso de esas bibliotecas van a ser resueltas en la etapa de linkeo. En el caso de los módulos las funciones a las que tienen acceso están definidas en /proc/kallsyms y las definiciones de esos símbolos pertenecientes al archivo objeto que es en sí el módulo se resuelven cuando se ejecuta insmod o modprobe.

b) Espacio de Usuario - Espacio de Kernel El kernel gestiona el acceso a recursos por parte de los procesos asegurando que no se haga un acceso a los mismos de forma indiscriminada (además de todas las otras tareas que tiene el kernel sobre los procesos como es protección, integridad, etc.). Las CPU (específicamente intel 80386 en este caso) tiene cuatro niveles de privilegio de los cuales los sistemas Unix usan dos: el de mayor privilegio como supervisor y el de menor como usuario (anillos 0 y 3 respectivamente). Respecto al espacio de usuario y espacio de kernel en general los programas que se usan cotidianamente trabajan a nivel de usuario y existen tres maneras básicas de pasar de nivel de usuario a nivel de kernel: a través de llamadas a sistema, por interrupciones o por excepciones. El primer caso es efectivamente un paso de usuario a kernel, en el caso de interrupciones o excepciones no necesariamente significa un cambio de modo ya que podría estar ejecutando código en kernel y manejar una interrupción o excepción como una entrada a un nuevo contexto de ejecución pero en el kernel todavía, lo que en sí implica que no hubo ningún cambio de modo.

c) Espacio de Datos El Espacio de Datos en la programación de módulos hace referencia a la región de memoria donde el kernel almacena sus variables globales, estructuras y buffers. A diferencia de los procesos de usuario que se ejecutan de forma aislada, el kernel y todos sus módulos comparten un único entorno de

memoria continua. Dentro de este entorno global conviven el Code Space, donde se ejecutan las instrucciones compartidas, y el Name Space, el cual exige que todos los símbolos y variables definidos en los datos sean únicos para evitar conflictos a nivel de todo el núcleo.

Dado que no existen barreras de protección de memoria en este espacio compartido, el manejo de los datos es fundamental. Un error en un módulo puede corromper las variables de otros componentes o del propio kernel, provocando la caída de todo el sistema. Siguiendo esta arquitectura, un módulo jamás puede acceder directamente a los datos de un programa de usuario, teniendo que utilizar siempre las funciones seguras de la API del kernel para transferir información entre ambos espacios.

d) Drivers Los drivers son un tipo de módulo que se encarga de hacer la conexión entre el archivo que define mi elemento de hardware (que no necesariamente es un componente exacto, es algo más abstracto porque por ejemplo en el caso de los discos se pueden ver archivos asociados a las particiones) en /dev y el elemento puntual. Esto permite a programas que quieran acceder por ejemplo a alguna característica asociada con el audio que directamente usen el archivo que se encuentra en /dev (/dev/audio) y el driver se encargue de la conexión con la placa de audio concreta. Si se lista el contenido de /dev con "ls -l" se pueden observar entre otras cosas dos números separados de una coma. El primer número es el número mayor y hace referencia al driver que controla al dispositivo, el segundo es el número menor y especifica qué hardware de todos los asociados al driver se está viendo. Al acceder a un dispositivo el kernel solo precisa del número mayor, esto deja en claro que es el controlador el que se ocupa del número menor, utilizándolo para diferenciar entre los distintos componentes de hardware. Volviendo nuevamente a la ejecución de "ls -l" para ver /dev hay algo importante a mencionar: existen dos tipos de archivos de dispositivos, de bloque y de carácter. Los de bloque tienen un búfer para las solicitudes, lo que les permite elegir el orden óptimo para responderlas y además solo pueden aceptar entrada y devolver salida en bloques a diferencia de los dispositivos de carácter que pueden utilizar tantos bytes como necesiten.

Consignas

1)¿Qué diferencias se pueden observar entre los dos modinfo?

En base a la imagen adjunta en la que se puede ver la salida de modinfo completa para "mimodulo.ko" y casi completa para "/lib/modules/\$(uname -r)/kernel/crypto/des_generic.ko.zst" la principal diferencia que se puede observar es la existencia de alias que en el caso del módulo hecho por nosotros no tiene. Además de la longitud de la firma a favor del módulo crypto que lamentablemente no se logra apreciar en esta captura se ven diferencias en las dependencias y en el hecho de que este módulo se distribuye junto con el kernel, es decir, es parte del mismo y eso se indica en "intree: Y" que en nuestro módulo no está.


```

mario@mario-IdeaPad-Gaming-3-15IAM7:~/Documentos/Ejercicios_Sist_Comp/TP4/kernel-modules/part1/modules$ modinfo mimodulo.ko
filename:      /home/mario/Documentos/Ejercicios_Sist_Comp/TP4/kernel-modules/part1/module/mimodulo.ko
author:        Catedra de SdeC
description:    Primer modulo ejemplo
license:        GPL
srcversion:     C6390617B2101FB1B600A9
depends:
retpoline:     Y
name:          mimodulo
vermagic:       6.8.0-117-generic SMP preempt mod_unload modversions
sig_id:         PKCS#7
signer:         MI clave para modulos
sig_key:        5C:96:1A:8B:16:CA:D1:24:D1:09:F8:C6:7D:16:15:6F:A1:5C:15:16
sig_hashalgo:   sha256
signature:      05:1B:55:DA:FE:A8:81:2A:04:42:9E:EB:AD:5C:54:A2:4F:44:14:AA:
6A:1C:59:80:81:78:10:DE:FA:D2:BE:EA:E8:93:20:70:A0:C2:3C:CD:
06:AD:97:67:C5:E2:23:20:F1:DE:F7:B8:2A:35:F1:80:87:50:41:08:
0A:CD:2F:93:51:A5:BF:15:50:BE:C3:23:C2:09:80:75:8C:07:91:37:
59:18:A0:87:17:16:D2:CC:93:F9:1B:71:E7:73:52:DA:80:DA:FD:83:
46:25:F9:30:A8:39:60:F3:07:93:31:72:99:86:46:0F:80:A0:F8:05:
55:95:4F:19:F3:8F:CA:ED:15:B6:F5:A9:5C:D2:FF:84:1F:50:E8:69:
4C:A4:69:D7:2E:34:79:EC:82:67:E7:7D:3C:F1:9E:0F:60:8C:10:91:
1C:EE:74:5C:67:28:36:60:F5:5F:52:34:1F:58:9B:80:37:83:1C:AF:
32:E5:7C:12:28:CF:A6:E5:8E:A0:4A:2F:60:9C:19:82:08:B8:49:40:
D9:52:B7:39:40:C1:39:A0:DC:90:88:BE:B3:8C:7C:3E:65:D8:60:A1:
F1:60:E4:14:C9:34:68:71:FB:B2:AC:87:06:74:7A:62:0F:3C:AE:16:
C0:A9:32:C8:D1:25:EF:16:42:44:38:13:00:12:76:6A

mario@mario-IdeaPad-Gaming-3-15IAM7:~/Documentos/Ejercicios_Sist_Comp/TP4/kernel-modules/part1/modules$ modinfo /lib/modules/$(uname -r)/kernel/crypto/des_g
eneric.ko.zst
filename:      /lib/modules/6.8.0-117-generic/kernel/crypto/des_generic.ko.zst
alias:         crypto-des3_edc-generic
alias:         des3_edc-generic
alias:         crypto-des3_edc
alias:         des3_edc
alias:         crypto-des-generic
alias:         des-generic
alias:         crypto-des
alias:         des
author:        Dag Arne Osvik <da@osvik.no>
description:    DES & Triple DES EDE Cipher Algorithms
license:        GPL
srcversion:     B56606AD918CF0074032808
depends:        libdes
retpoline:     Y
intree:        Y
name:          des_generic
vermagic:       6.8.0-117-generic SMP preempt mod_unload modversions
sig_id:         PKCS#7
signer:         Build time autogenerated kernel key
sig_key:        60:04:C4:BF:F4:02:05:5D:8D:2A:6A:72:72:44:11:FF:31:09:80
sig_hashalgo:   sha512
signature:      5B:34:93:6A:54:A2:A1:FF:0E:D0:4A:80:58:E0:1F:90:87:8F:A4:0F:
E0:3E:8B:87:66:16:08:74:5E:31:D1:B6:48:A1:27:0B:12:7F:0F:DA:
26:51:DC:22:BA:1B:6F:1F:9A:28:B1:D8:A8:E9:40:20:27:23:19:3E:
5E:6F:D2:41:7E:0A:1A:7A:58:E9:6A:4D:9B:80:D0:03:60:AB:92:C9:
57:F6:A2:0F:0C:E8:C1:8C:D9:85:B7:B3:EA:93:E4:12:0B:3F:99:84:
17:A8:F0:74:6F:FD:08:F2:0F:51:C5:0E:97:72:48:05:E5:27:BF:84:
A4:AB:6E:8A:90:0D:AC:15:8D:57:01:95:F2:32:91:47:FF:B7:29:48:

```

2) ¿Qué divers/modulos están cargados en sus propias pc?

Para poder hacer esto, ejecutamos el siguiente comando con el que se carga toda la lista de los modulos/drivers en nuestra carpeta outputs: `lsmod > outputs/modulos_cargados0.txt` Y para ver las diferencias utilizamos la pagina [diff checker](#) cuya salida es la siguiente:

The image is a composite of two screenshots. The left screenshot shows the 'Diffchecker Desktop' application. It has a 'Compare text' section with a 'Diffchecker Desktop' logo and a description: 'The most secure way to run Diffchecker. Get the Diffchecker Desktop app on your computer!'. Below this, there's a 'Tools' section with 'Real-time editor', 'Hide unchanged lines', and 'Disable line wrap'. The main area shows a comparison of two files, with a table of results:

Module	Size	Used by
1. Module		
2. rfcmm	102400	16
3. snd_seq_dummy	12288	0
4. snd_hrtimer	12288	1
5. ccw	20480	0
6. xe	2727936	0
7. snd_ctl_led	24576	0
8. snd_ctl_led	12288	1
9. snd_hda_codec_realtek	208896	1
10. snd_hda_codec_generic	122880	1

The right screenshot shows a video player interface. It has a 'Transiciones de escena' (Scene Transitions) section with a list of scenes: 'Escena 4', 'discord', 'Escena 3', 'Escena 2', and 'Escena 5'. Below this, there's a 'Fuentes' (Sources) section with 'Audio C. J. J. J.' and 'Audio C. J. J. J.'. The main area shows a 'Mezclador de audio' (Audio Mixer) with various sliders and buttons. The bottom of the image shows a Windows taskbar with various icons and a system tray.

3) ¿Cuáles no están cargados pero están disponibles? que pasa cuando el driver de un dispositivo no está disponible.

Todos aquellos módulos que se encuentren en `/lib/modules/$(uname -r)` son los que están disponibles y los que se pueden ver en `lsmod` (o en su defecto `cat /proc/modules`) son los cargados en el momento. En el caso de que un driver de dispositivo esté en `/lib/modules/$(uname -r)` y no se haya levantado se puede hacer manualmente o forzar la carga automática para que aparezca luego en `lsmod`, en caso de ni siquiera existir el `.ko` el dispositivo queda sin drivers asociado y por ende no puede ser utilizado hasta que se instale el paquete necesario.

4) Correr `hwinfo` en una pc real con hw real y agregar la url de la información de hw en el reporte.

URL generada : <https://linux-hardware.org/?probe=9b15e49952>

5) ¿Qué diferencia existe entre un módulo y un programa?

Un programa se ejecuta en espacio de usuario, usa funciones de bibliotecas ya sea de `libc` o alguna otra especializada especificada por el programador y tiene una función `main` principal. Un módulo se ejecuta en espacio de kernel, accede a funciones definidas en `kallsyms` y no tiene una función `main`, su estructura se basa principalmente en `module_init` y `module_exit`.

6) ¿Cómo puede ver una lista de las llamadas al sistema que realiza un simple `helloworld` en c?

Suponiendo un programa sencillo que imprime `hello` se compila al mismo con `gcc -Wall -o hello hello.c` y se corre el ejecutable con `strace ./hello`. Esto muestra en consola todas las llamadas al sistema que el proceso y sus hijos realizan.

7) ¿Qué es un segmentation fault? ¿Cómo maneja el kernel y como lo hace un programa?

Un segmentation fault es una violación de acceso a memoria detectada por el hardware (la MMU en sí). Este fallo se lanza cuando un programa intenta leer, escribir o ejecutar una dirección de memoria que no está mapeada en su espacio de direcciones, acceder a una página con permisos insuficientes (escribir en una página de solo lectura, ejecutar código en una página no ejecutable) o usar direcciones en el rango reservado del kernel desde espacio de usuario.

En caso de que un programa en espacio de usuario cometa un error el manejador de excepciones del kernel envía un `SIGSEGV` al proceso que causó la falta y el kernel y los demás procesos siguen funcionando sin problema.

Para el caso del kernel cuando la MMU detecta la violación la CPU genera la excepción, el manejador del kernel examina la dirección que falló y el contexto. Como el fallo provino del propio código del kernel se invoca la función `__die()` (o variantes) que imprime un `Oops`, i.e., un volcado de los registros, la pila de llamadas y el estado del proceso actual. El kernel muere y se requiere un reinicio.

8 y 9) Intento de firmar un módulo de kernel : Documentación y evidencia.

```

1 #include <linux/module.h>
2 #include <linux/kernel.h>
3 MODULE_LICENSE("GPL");
4 MODULE_DESCRIPTION("Module TMNC");
5 MODULE_AUTHOR("TeaM@rgenConflict");
6
7 int module_init(void)
8 {
9     printk(KERN_INFO "TMNC saluda desde kernel!!\n");
10    return 0;
11}
12
13 void module_cleanup(void)
14 {
15     printk(KERN_INFO "TMNC se despide del kernel!!\n");
16}
17
18 module_init(module_init);
19 module_exit(module_cleanup);

```

```

mario@maria-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$ make -C /lib/modules/6.8.0-117-generic/build M=/home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_P
MODULO_PROPIO
make: se entra en el directorio '/usr/src/linux-headers-6.8.0-117-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: gcc-13 (Ubuntu 13.3.0-2ubuntu2-24.04.1) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-2ubuntu2-24.04.1) 13.3.0
CC [M] /home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/tmc.o
/home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/tmc.c:17:5: warning: no previous prototype for 'module_init' [-Wmissing-prototypes]
7 | int module_init(void)
  | ^
/home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/tmc.c:13:6: warning: no previous prototype for 'module_cleanup' [-Wmissing-prototypes]
13 | void module_cleanup(void)
    | ^
MODPOST /home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/Module.symvers
CC [M] /home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/tmc.mod.o
LD [M] /home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/tmc.ko
BTF [M] /home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/tmc.ko
Skipping BTF generation for /home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/tmc.ko due to unavailability of vmlinux
make: se sale del directorio '/usr/src/linux-headers-6.8.0-117-generic'
mario@maria-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$

```

Lo primero que se hace es ejecutar “make -C /lib/modules/6.8.0-117-generic/build M=/home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO modules” donde “6.8.0-117-generic” es el reemplazo efectivo de “uname -r”.

```

mario@maria-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$ make -C /lib/modules/6.8.0-117-generic/build M=/home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_P
MODULO_PROPIO
MODPOST /home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/Module.symvers
CC [M] /home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/tmc.mod.o
LD [M] /home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/tmc.ko
BTF [M] /home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/tmc.ko
Skipping BTF generation for /home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/tmc.ko due to unavailability of vmlinux
make: se sale del directorio '/usr/src/linux-headers-6.8.0-117-generic'
mario@maria-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$ sudo openssl req -new -x509 -newkey rsa:2048 -keyout MOK.priv -outform DER -out MOK.der -nodes -days 36
500 -subj "/CN=Clave para modulo TMNC/"
[sudo] contraseña para mario:
.....
mario@maria-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$ sudo /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ./MOK.priv ./MOK.der tmc.ko

```

Seguido se procede con la creación de una clave privada (MOK.priv) y un certificado público (MOK.der) para firma y verificación de módulo con “sudo openssl req -new -x509 -newkey rsa:2048 -keyout MOK.priv -outform DER -out MOK.der -nodes -days 36500 -subj “/CN=Clave para modulo TMNC”/”; importante mencionar que en este paso se pide crear una clave que es necesaria más adelante. Luego se firma el módulo con “sudo /usr/src/linux-headers-uname -r/scripts/sign-file sha256 ./MOK.priv ./MOK.der mimodulo.ko” indicando que se usa cifrado sha256.

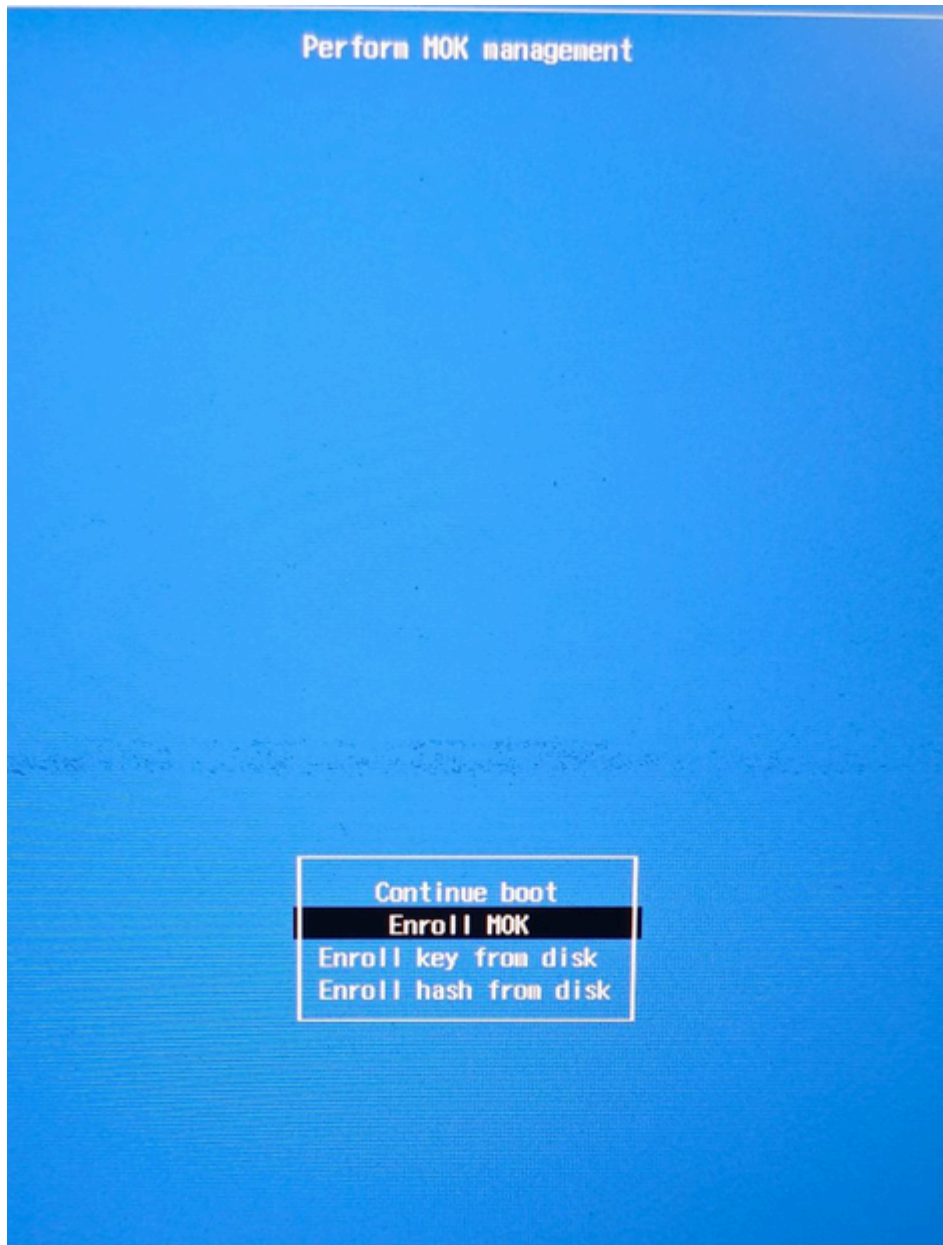
```

mario@maria-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$ sudo /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ./MOK.priv ./MOK.der tmc.ko
mario@maria-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$ sudo mokutil --import MOK.der

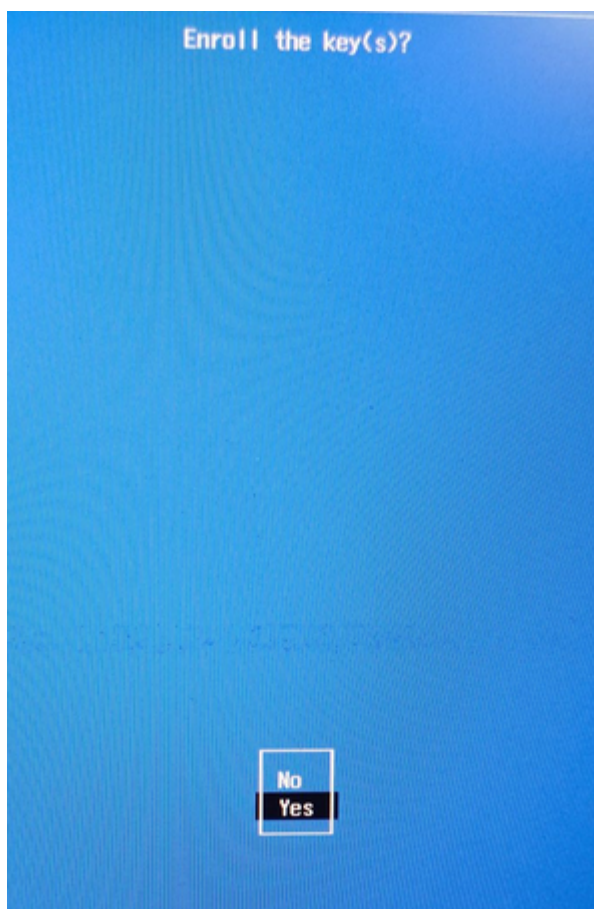
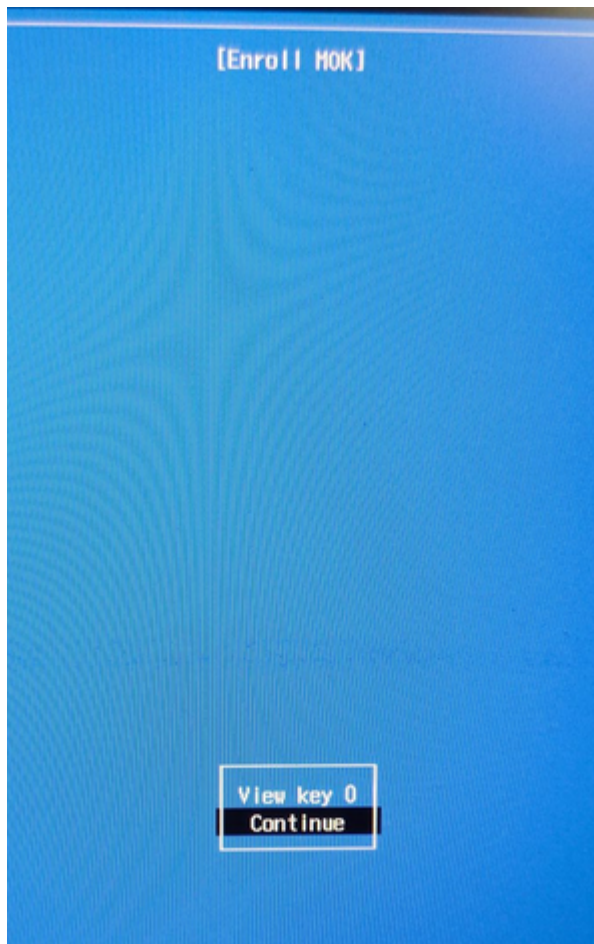
```

Para que el Arranque Seguro acepte el módulo es necesario inscribir el certificado público en la lista de Claves del Propietario de la Máquina (MOK) con “sudo mokutil --import MOK.der”.

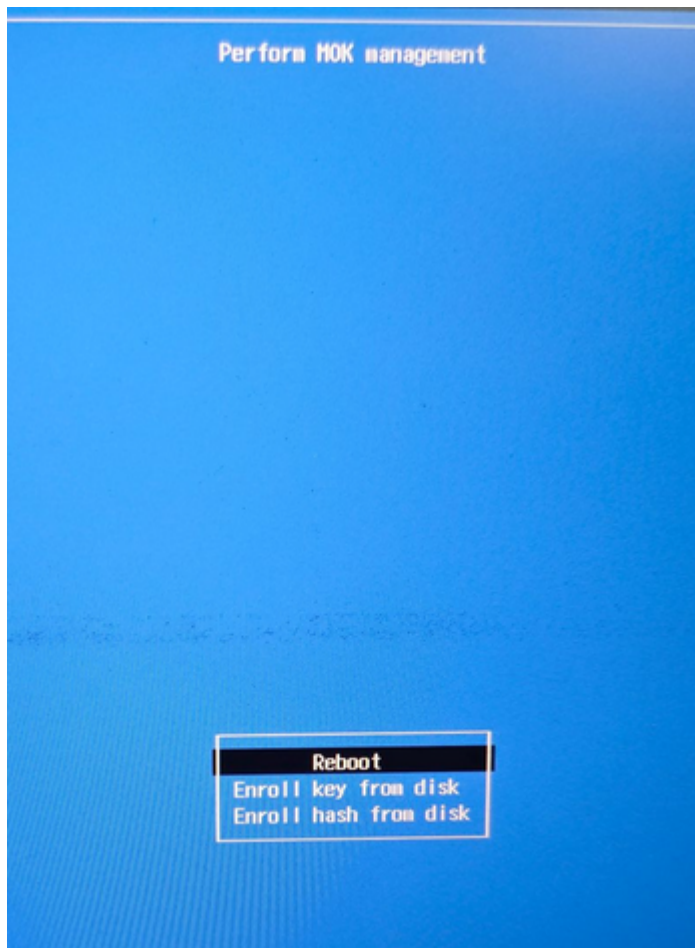
Una vez realizado todo esto se procede a reiniciar la PC e interactuar con el MOK management.



Se selecciona "Enroll MOK". Algunos de los pasos subsiguientes no van a ser dejados por escrito porque la imagen es lo suficientemente explicativa.



En este paso intermedio es necesario introducir la clave que se había definido previamente, a continuación de ello se debe hacer reboot.



Una vez cumplido todo esto es posible cargar el módulo y firmado con “sudo insmod tmnc.ko”. Para este ejemplo se introduce el módulo y se retira inmediatamente para mostrar la entrada y la salida del mismo en los registros de kernel con “sudo dmesg | tail” donde “| tail” es usado para mostrar solo la cola (o parte final) de todo el mensaje que se imprime por consola.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• mario@mario-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$ sudo insmod tmnc.ko
• mario@mario-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$ sudo rmmod tmnc
• mario@mario-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$ sudo dmesg | tail
[ 61.962746] input: Logitech Wireless Mouse PID:4074 Mouse as /devices/pci0000:00/0000:00:14.0/usb3/3-1/3-1:1.2/0003:046D:C53F.000A/0003:046D:4074.000B/input/input35
[ 61.963032] hid-generic 0003:046D:4074.000B: input,hidraw7: USB HID v1.11 Keyboard [Logitech Wireless Mouse PID:4074] on usb-0000:00:14.0-1/input2:1
[ 62.081955] input: Logitech G305 as /devices/pci0000:00/0000:00:14.0/usb3/3-1/3-1:1.2/0003:046D:C53F.000A/0003:046D:4074.000B/input/input39
[ 62.114864] logitech-hidpp-device 0003:046D:4074.000B: input,hidraw7: USB HID v1.11 Keyboard [Logitech G305] on usb-0000:00:14.0-1/input2:1
[ 62.704704] logitech-hidpp-device 0003:046D:4074.000B: HID++ 4.2 device connected.
[ 94.754619] tmnc: loading out-of-tree module taints kernel.
[ 94.755744] TMNC saluda desde kernel!!
[ 108.022413] TMNC se despide del kernel :(
[ 170.661913] TMNC saluda desde kernel!!
[ 183.766531] TMNC se despide del kernel :(
mario@mario-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$

```

```

PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS
• mario@mario-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$ sudo insmod tmnc.ko
• mario@mario-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$ lsmod | grep tmnc
tmnc                12288  0
• mario@mario-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$ modinfo tmnc.ko
filename:           /home/mario/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO/tmnc.ko
author:             TeamMergeNoConflict
description:        Modulo TMNC
license:            GPL
srcversion:         7A0E737A27C5E421B7BE697
depends:
retpoline:         Y
name:              tmnc
vermagic:          6.8.0-117-generic SMP preempt mod_unload modversions
sig_id:            PKCS#7
signer:            Clave para modulo TMNC
sig_key:           49:22:B0:CB:45:90:6C:B3:0E:BC:43:6A:B0:4C:EA:27:09:FE:FE:BD
sig_hashalgo:      sha256
signature:         B2:93:30:C7:CC:68:E0:F1:0E:C3:9F:1A:B1:12:EF:E4:15:11:8E:94:
                    50:50:6E:F9:9B:AE:FC:54:F8:20:55:E6:E0:FC:1B:C0:F6:A3:0D:62:
                    ED:BC:64:A4:AC:43:B2:DB:E9:86:75:B6:CD:EF:EF:0C:5A:3E:1B:9B:
                    03:25:3A:CE:3D:42:6C:C2:26:A7:76:35:53:BA:00:2E:78:04:C0:69:
                    D2:08:54:EB:0F:B9:AC:71:CE:B6:99:DF:7E:B4:86:F8:4C:6D:3C:4B:
                    95:6B:21:4D:F4:E2:A2:17:80:D8:50:4D:3A:1D:CA:9F:55:FA:52:89:
                    6C:55:4B:23:AA:2D:90:6C:C7:54:93:34:6D:F8:4B:5A:4A:1B:75:EE:
                    52:DE:FB:A8:DF:B0:87:9A:B2:C1:A0:DF:84:5F:E6:35:AE:1D:AD:F8:
                    D5:9B:F5:9D:3F:94:64:3B:4E:A3:50:ED:69:02:9C:39:44:B9:7B:9D:
                    D6:D4:BC:68:26:3D:6F:94:EC:2B:4D:6F:1D:3D:BE:3D:98:D8:D0:C0:
                    96:FB:A8:2A:97:AC:5F:24:44:3C:30:30:F0:07:09:E7:EB:C4:CA:FF:
                    A4:4C:B4:21:47:25:6F:4A:D9:B7:09:C6:99:FA:2E:E6:A1:E9:AB:D4:
                    6C:78:A2:94:86:61:64:84:88:16:16:FA:38:AD:09:71
• mario@mario-IdeaPad-Gaming-3-15IAH7:~/Documentos/Ejercicios_Sist_Comp/TP4/MODULO_PROPIO$

```

Finalmente se carga una última vez para poder acceder desde lsmod (para modinfo no es necesario insertar el módulo).

10) ¿Que pasa si mi compañero con secure boot habilitado intenta cargar un módulo firmado por mi?

Si mi compañero tiene Secure Boot activado, su sistema solo permite cargar módulos del kernel que estén firmados digitalmente con una clave reconocida como segura. Esto forma parte de una medida de seguridad que evita la ejecución de código malicioso a bajo nivel (como rootkits).

Si yo firme mi módulo con una clave propia (que no está registrada en su sistema), el kernel va a rechazar la carga del módulo, incluso si está correctamente firmado. Mostrará un error indicando que la firma no es válida o que falta la clave.

Para que pueda cargarlo, tendría que importar mi clave pública al sistema usando herramientas como mokutil, y luego autorizarla manualmente al reiniciar, para que Secure Boot la reconozca como confiable.

Con Secure Boot activo, no alcanza con firmar el módulo, también es necesario que la clave usada esté autorizada por el sistema.

11) Basado en la siguiente nota <https://arstechnica.com/security/2024/08/a-patch-microsoft-spent-2-years-preparing-is-making-a-mess-for-some-linux-users/>:

- **¿Cuál fue la consecuencia principal del parche de Microsoft sobre GRUB en sistemas con arranque dual (Linux y Windows)?**

Los sistemas ya no podían iniciar Linux debido a una violación de política de seguridad del Secure Boot.

- **¿Qué implicancia tiene desactivar Secure Boot como solución al problema descrito en el artículo?**

Desactivar Secure Boot implica comprometer la seguridad del sistema, ya que Secure Boot impide la carga de código no firmado durante el arranque. Al desactivarlo para permitir la carga de controladores o módulos que el parche de Microsoft bloquea, se abre la posibilidad de que malware o rootkits se carguen sin restricciones al iniciar el sistema.

- **¿Cuál es el propósito principal del Secure Boot en el proceso de arranque de un sistema?**

Verificar que solo se ejecuten binarios firmados y autorizados durante el arranque.